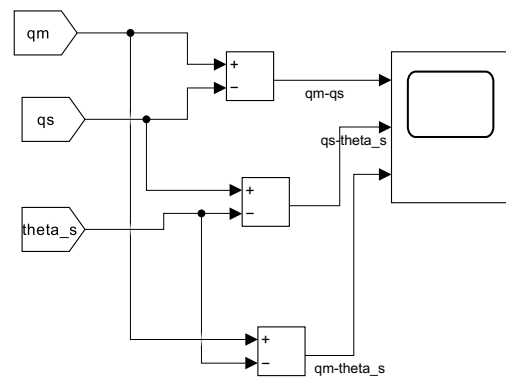
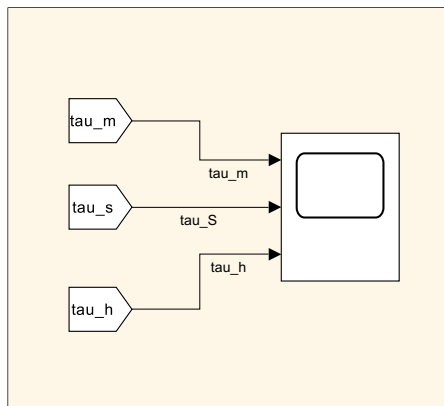
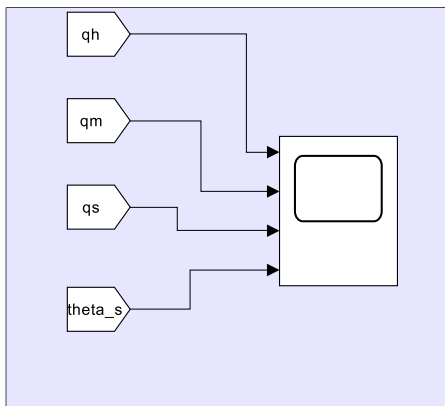
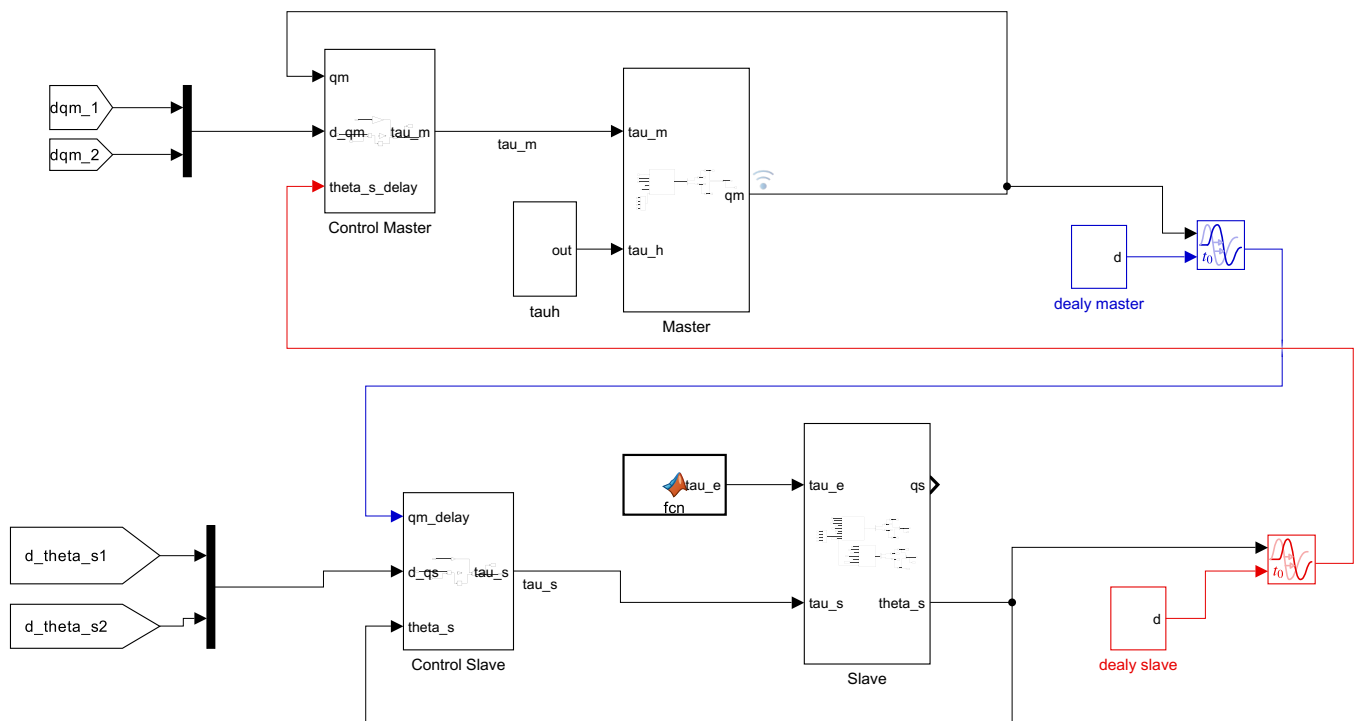
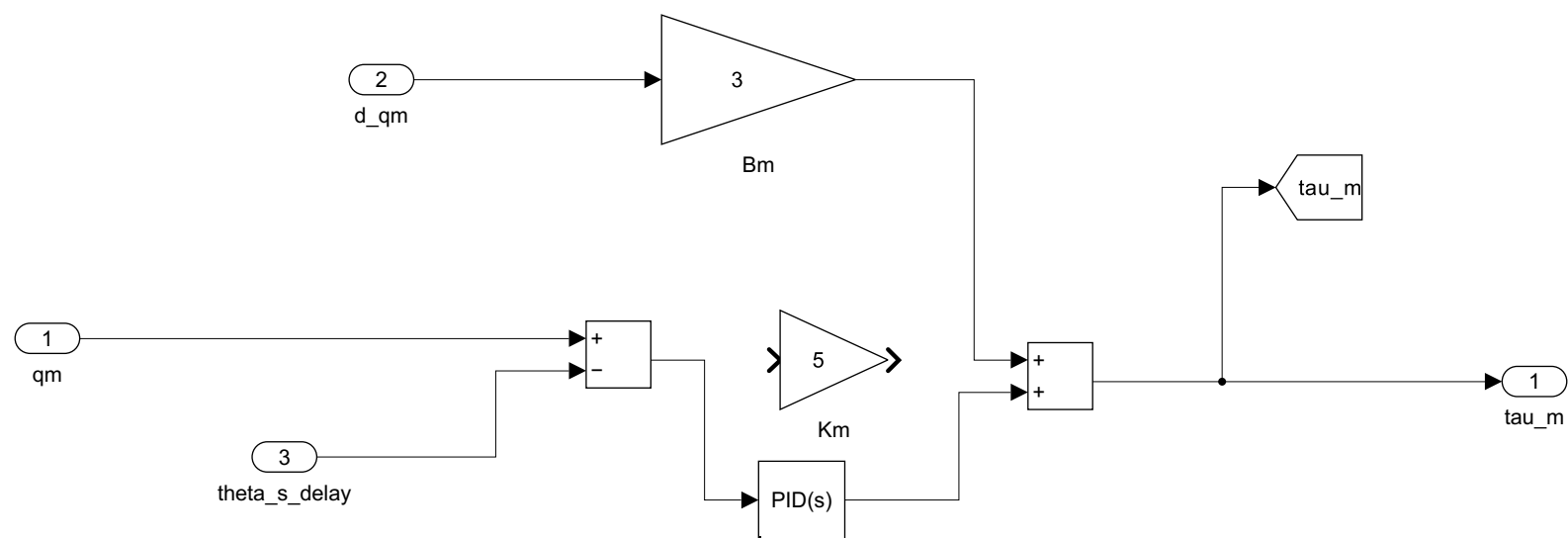
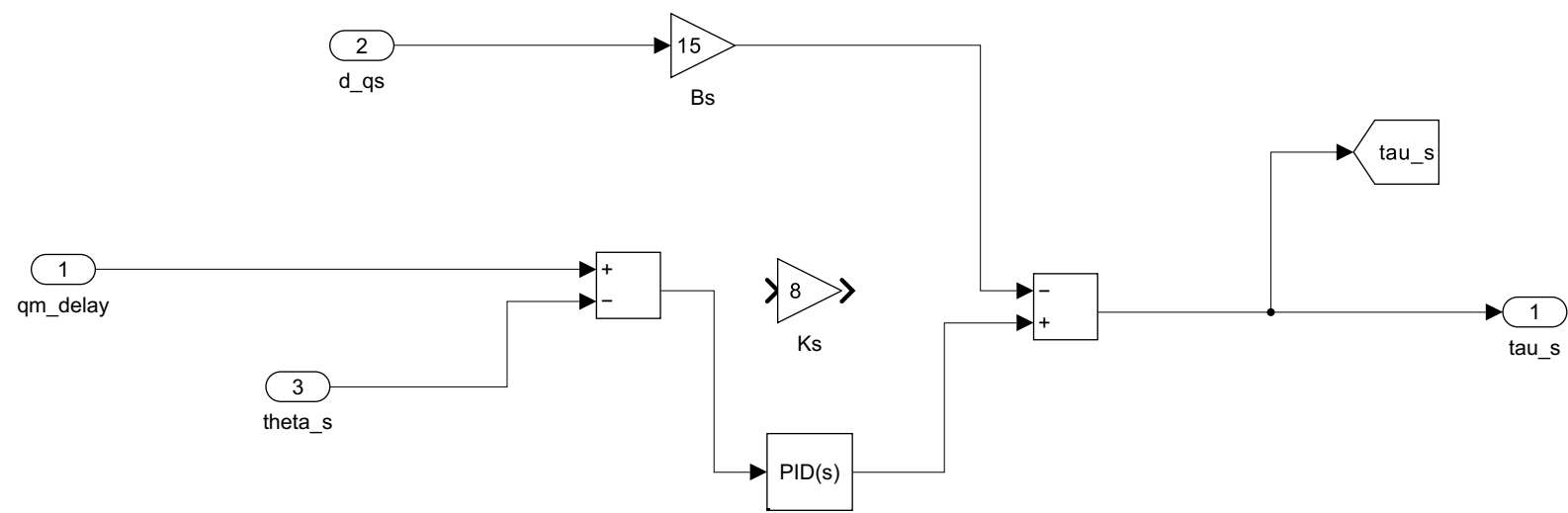


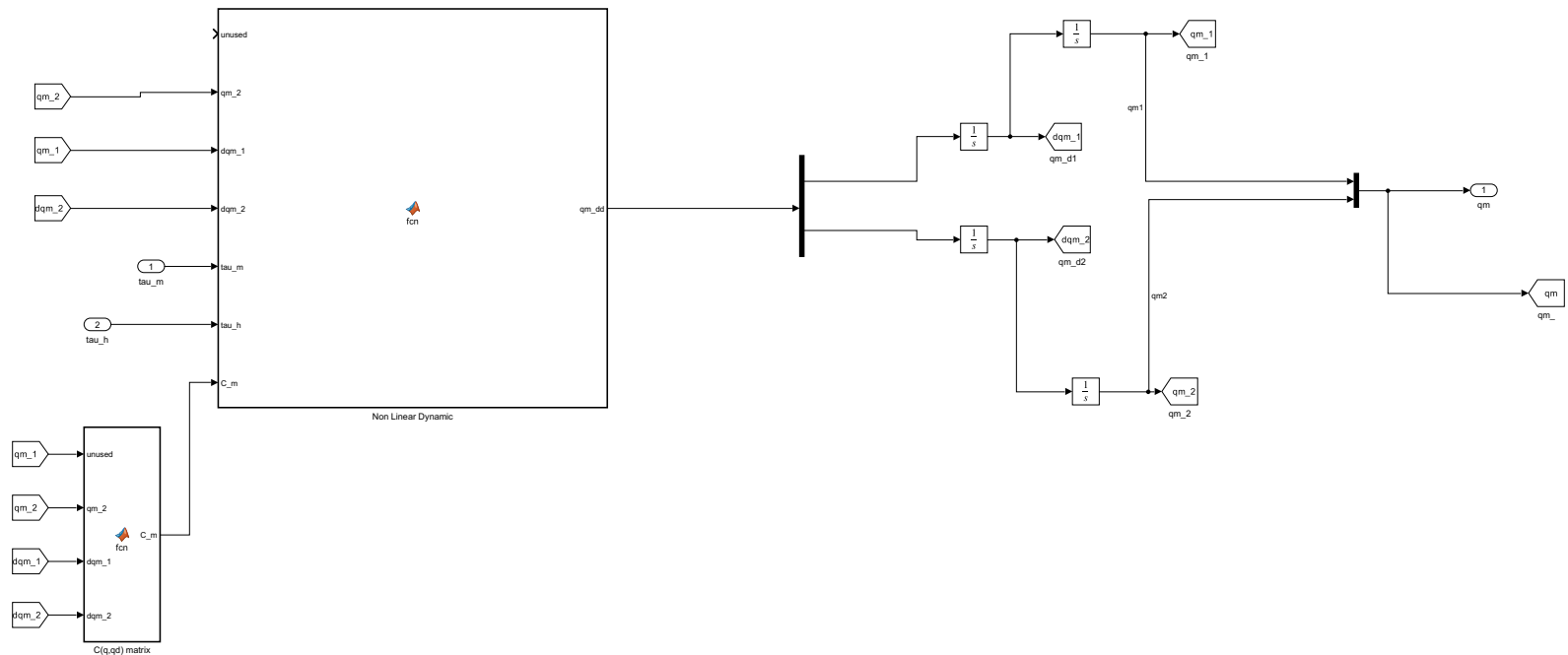






```
function tau_e = fcn()
```

```
tau_e=[0;0];
```



```
function C_m = fcn(~,qm_2,dqm_1,dqm_2)
```

```
% Physiscal Parameters
```

```
Lm1 = 0.38;
```

```
Lm2 = 0.38;
```

```
mm1 = 3.5;
```

```
mm2 = 0.5;
```

```
%Master_first_link_length
```

```
%Master_second_link_length
```

```
%Master_first_link_mass
```

```
%Master_second_link_mass
```

```
betha_m=Lm1*Lm2*mm2;
```

```
C_m_11 = -2*betha_m*sin(qm_2)*dqm_2;
```

```
C_m_12 = -betha_m*sin(qm_2)*dqm_2;
```

```
C_m_21 = betha_m*sin(qm_2)*dqm_1;
```

```
C_m_22 = 0;
```

```
C_m = [C_m_11 C_m_12; C_m_21 C_m_22];
```

```
end
```

```
function qm_dd =fcn(~,qm_2,dqm_1,dqm_2,tau_m,tau_h,C_m)
```

```
% Physcal Parameters
```

```
Lm1 = 0.38;
```

```
Lm2 = 0.38;
```

```
mm1 = 3.5;
```

```
mm2 = 0.5;
```

```
%Master_first_link_length
```

```
%Master_second_link_length
```

```
%Master_first_link mass
```

```
%Master_second_link mass
```

```
dqm=[dqm_1;dqm_2];
```

```
alpha_m=(Lm2^2)*mm2+(Lm1^2)*(mm1+mm2);
```

```
betha_m=Lm1*Lm2*mm2;
```

```
delta_m=(Lm2^2)*mm2;
```

```
M_m_11= alpha_m + 2*betha_m*cos(qm_2);
```

```
M_m_12= delta_m + betha_m * cos(qm_2);
```

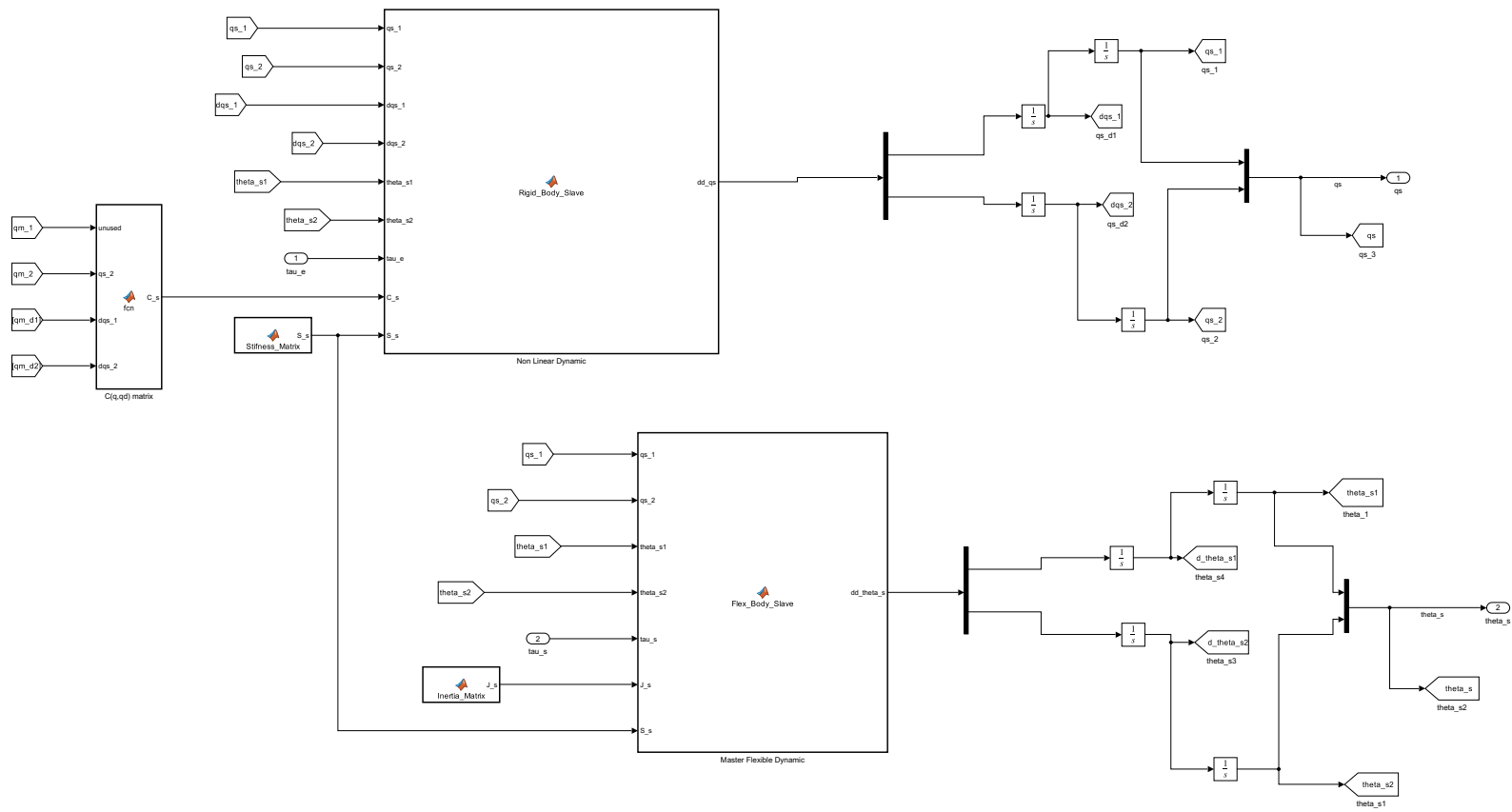
```
M_m_21= M_m_12;
```

```
M_m_22= delta_m;
```

```
M_m = [M_m_11 M_m_12; M_m_21 M_m_22];
```

```
qm_dd = inv(M_m)*( -tau_m -(C_m * dqm ) + tau_h );
```

```
end
```



```
function C_s = fcn(~,qs_2,dqs_1,dqs_2)
```

```
% Physiscal Parameters
```

```
Ls1 = 0.38;
```

```
Ls2 = 0.38;
```

```
ms1 = 0.5;
```

```
ms2 = 0.35;
```

```
betha_s=Ls1*Ls2*ms2;
```

```
C_s_11 = -2*betha_s*sin(qs_2)*dqs_2;
```

```
C_s_12 = -betha_s*sin(qs_2)*dqs_2;
```

```
C_s_21 = betha_s*sin(qs_2)*dqs_1;
```

```
C_s_22 = 0;
```

```
C_s= [C_s_11 C_s_12; C_s_21 C_s_22];
```

```
end
```

```
%Slave_first_link_length
```

```
%Slave_second_link_length
```

```
%Slave_first_link_mass
```

```
%Slave_second_link_mass
```

```
function S_s = Stifness_Matrix()
    % This function creates a 2x2 identity matrix scaled by 500

    % Create a 2x2 identity matrix
    I = eye(2);

    % Scale the identity matrix by 500
    S_s = 500 * I;
end
```

```
function J_s = Inertia_Matrix()

    % Create a 2x2 identity matrix
    I = eye(2);
    % Scale the identity matrix by 500
    J_s = 0.2 * I;
end
```

```
function dd_theta_s = Flex_Body_Slave(qs_1,qs_2,theta_s1,theta_s2,tau_s,J_s,S_s)

qs=[qs_1;qs_2];
theta_s=[theta_s1;theta_s2];

dd_theta_s = inv(J_s)*(tau_s - S_s*(theta_s-qs));

end
```

```

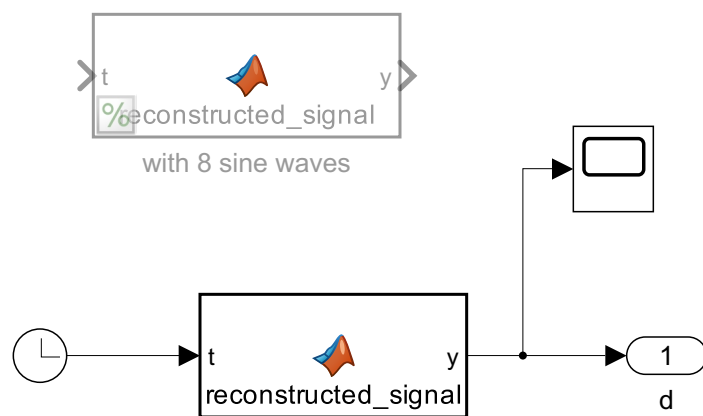
function dd_qs =Rigid_Body_Slave(qs_1,qs_2,dqs_1,dqs_2,theta_s1,theta_s2,tau_e,C_s,S_s)

% Physiscal Parameters
Ls1 = 0.38; %Slave_first_link_length
Ls2 = 0.38; %Slave_second_link_length
ms1 = 0.5; %Slave_first_link_mass
ms2 = 0.35; %Slave_second_link_mass

qs=[qs_1;qs_2];
theta_s=[theta_s1;theta_s2];
dqs=[dqs_1;dqs_2];
alpha_s=(Ls2^2)*ms2+(Ls1^2)*(ms1+ms2);
betha_s=Ls1*Ls2*ms2;
delta_s=(Ls2^2)*ms2;

M_s_11= alpha_s + 2*betha_s*cos(qs_2);
M_s_12= delta_s + betha_s*cos(qs_2);
M_s_21= M_s_12;
M_s_22= delta_s;
M_s = [M_s_11 M_s_12; M_s_21 M_s_22];
dd_qs= inv(M_s)*(- tau_e - (C_s*dqs) - (S_s*(qs-theta_s)));
end

```



```
function y = reconstructed_signal(t)
% Given data
DC_offset = 0.9613462407597305;
sine_amplitudes = [-0.02000659, -0.02349457, 0.00067474, 0.00159032, 0.00129016, -0.00135477, -0.00160663, -0.00160663];
cosine_amplitudes = [-0.02556397, 0.00393168, -0.0052079, 0.00220769, 0.00300956, 0.00160663, -0.00160663, -0.00160663];
frequencies = [0.24299065416927243, 0.48598130833854486, 0.7289719625078173, 0.9719626166770897, 1.2149352310151716, 1.4569063425232539, 1.6988774540313362, 1.9408485655394185];

% Number of terms
N = length(sine_amplitudes);

% Initialize the signal
y = DC_offset * ones(size(t));

% Add sine and cosine terms
for k = 1:N
    y = y + sine_amplitudes(k) * sin(2 * pi * frequencies(k) * t) + cosine_amplitudes(k) * cos(2 * pi * frequencies(k) * t);
end
end
```

```
function y = reconstructed_signal(t)
% Given data
DC_offset = 0.9613888244716796;
sine_amplitudes = [-0.02002779, -0.02351674, 0.00074362, 0.00154452, 0.0013555, -0.00135695, -0.00135695, -0.00135695];
cosine_amplitudes = [-0.02562599, 0.00393932, -0.00523797, 0.0022634, 0.00295895, 0.00158273, -0.00158273, -0.00158273];
frequencies = [0.24299065416927243, 0.48598130833854486, 0.7289719625078173, 0.9719626166770897, 1.2149352311141716, 1.4579003916712539, 1.7008655522283362, 1.9438307127854185];

% Number of terms
N = length(sine_amplitudes);

% Initialize the signal
y = DC_offset * ones(size(t));

% Add sine and cosine terms
for k = 1:N
    y = y + sine_amplitudes(k) * sin(2 * pi * frequencies(k) * t) + cosine_amplitudes(k) * cos(2 * pi * frequencies(k) * t);
end
end
```

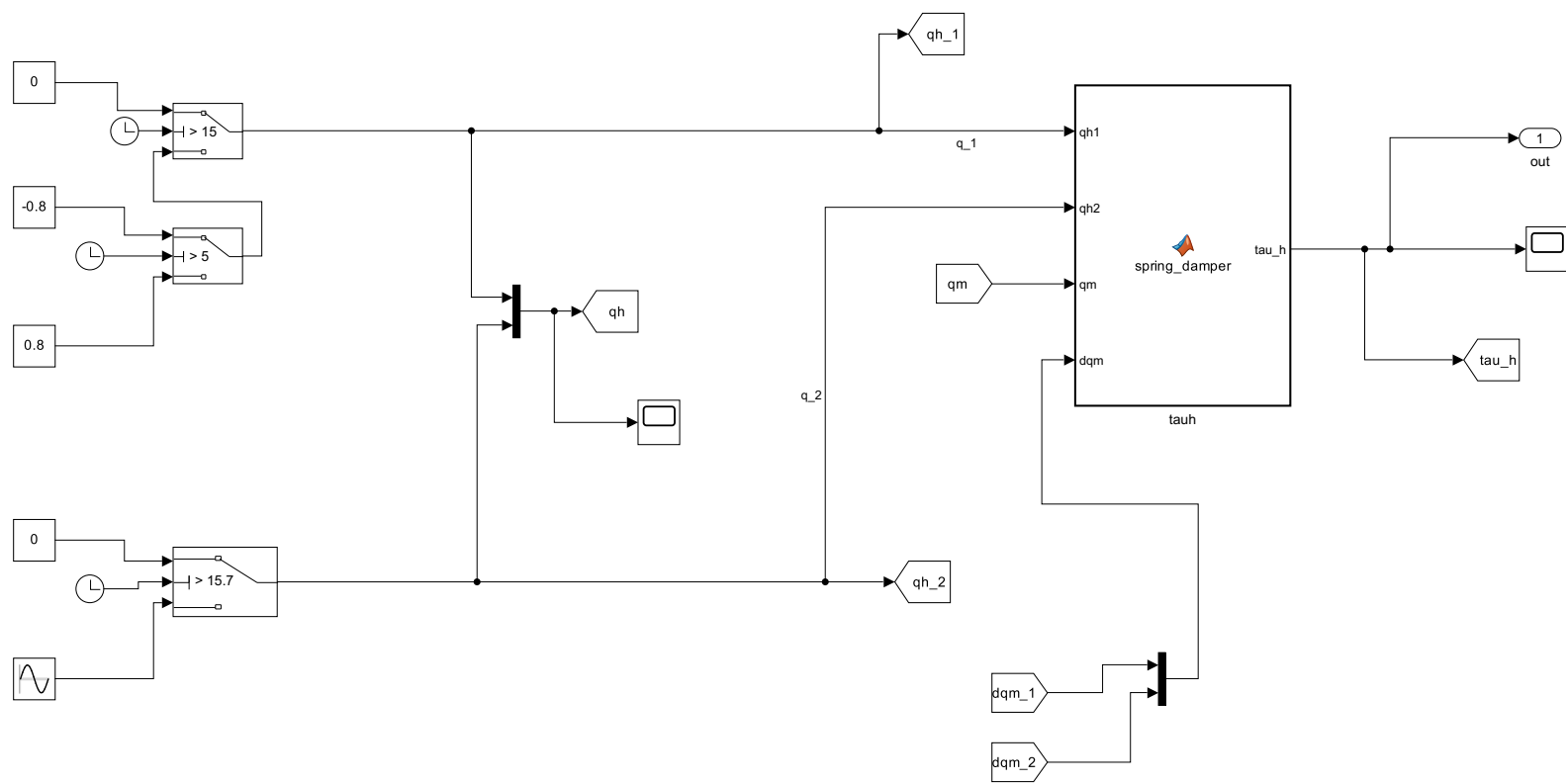


```
function y = reconstructed_signal(t)
% Given data
DC_offset = 0.9613462407597305;
sine_amplitudes = [-0.02000659, -0.02349457, 0.00067474, 0.00159032, 0.00129016, -0.00135477, -0.00160663, -0.00160663];
cosine_amplitudes = [-0.02556397, 0.00393168, -0.0052079, 0.00220769, 0.00300956, 0.00160663, -0.00160663, -0.00160663];
frequencies = [0.24299065416927243, 0.48598130833854486, 0.7289719625078173, 0.9719626166770897, 1.2149352310151716, 1.4569063425232539, 1.6988774540313362, 1.9408485655394185];

% Number of terms
N = length(sine_amplitudes);

% Initialize the signal
y = DC_offset * ones(size(t));

% Add sine and cosine terms
for k = 1:N
    y = y + sine_amplitudes(k) * sin(2 * pi * frequencies(k) * t) + cosine_amplitudes(k) * cos(2 * pi * frequencies(k) * t);
end
end
```



```
function tau_h = spring_damper(qh1,qh2,qm,dqm)
```

```
Kh=10;
```

```
Dh=2;
```

```
qh=[qh1;qh2];
```

```
tau_h=Kh*(qh-qm)-Dh*dqm;
```

```
end
```